

Digital Image Processing

Final Exam Suggestion — Detailed Answers

North Bengal International University (NBIU)

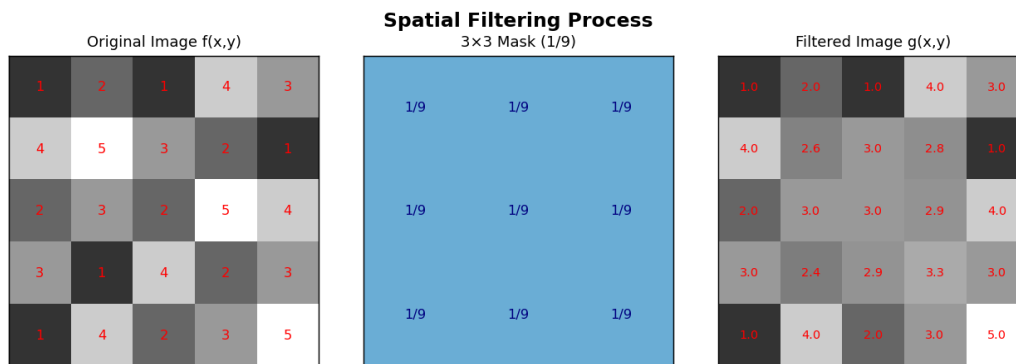
Q1. Define Spatial Filtering

Spatial filtering is a technique in image processing where we modify an image by applying a **filter** directly to the pixel values in the **spatial domain**. A spatial filter consists of a neighborhood and a predefined operation (e.g., averaging, finding median, or calculating derivatives) that is performed on the pixels within that neighborhood.

The process can be described as sliding the filter mask over every pixel of the image. At each position, the corresponding pixels under the mask are multiplied by the filter coefficients, summed, and the result becomes the new pixel value at the center of the mask.

Mathematically: $g(x, y) = \sum \sum w(s, t) \cdot f(x+s, y+t)$

Diagram — Spatial Filtering Process



Q2. Describe Averaging Filters & Effect of Mask Size in image enhancement.

Averaging Filter (Box Filter): An averaging filter is a linear smoothing filter used for blurring and noise reduction. It replaces the value of each pixel with the average of the pixel values in its neighborhood. All coefficients in the mask are equal. For example, a 3×3 averaging mask has all values $1/9$. The operation is:

$$g(x, y) = \frac{1}{mn} \sum_{s=-a}^a \sum_{t=-b}^b f(x+s, y+t)$$

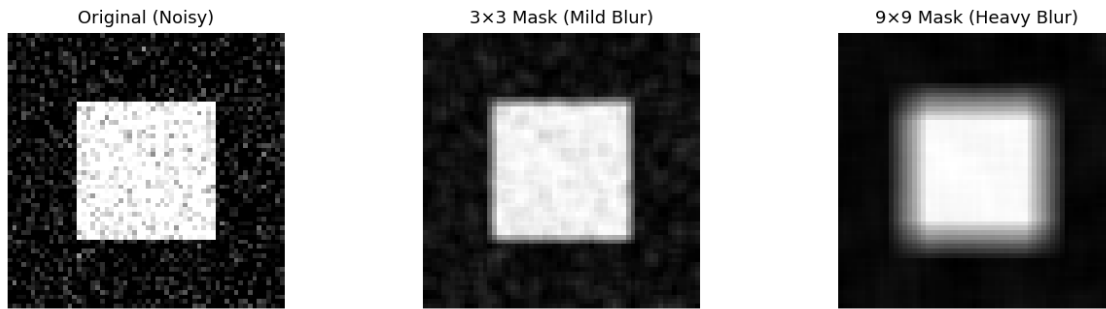
$$g(x, y) = (1/mn) \times \sum \sum f(x+s, y+t) \quad \text{where } mn = \text{total pixels in mask}$$

Effect of Mask Size

The size of the mask (e.g., 3×3 , 5×5 , 7×7) has a direct impact on the enhancement:

- **Small Mask (3×3):** Provides mild blurring and noise reduction. The image is slightly smoothed, and fine details are mostly preserved.
- **Large Mask (7×7 or larger):** Produces significant blurring. More noise is removed, but fine details (like edges and small objects) are also lost, making the image appear more "out of focus." Large masks are effective for removing high-intensity noise but cause severe edge degradation.

Effect of Averaging Mask Size



Q3. Why Median Filter is Better than Mean Filter?

Median Filter is normally used to reduce noise in an image, similar to the mean filter. However, it often does a better job than the mean filter in preserving useful details in an image.

Median filter has **2 main advantages**:

1. More Robust Against Noisy Pixels

The median is a more robust average than the mean. This means if a single pixel in a neighborhood has an unusual or extreme value (due to noise), it will not significantly affect the median value. So one bad pixel cannot ruin the output.

2. Better at Preserving Sharp Edges

Since the output of a median filter must always be the actual value of one of the pixels in the neighborhood, it never creates new or unrealistic pixel values. So when the filter moves over an edge in the image, that edge stays sharp and does not get blurred. This is why the median filter is much better at preserving sharp edges compared to the mean filter.

Note:

- Mean filter is better at dealing with Gaussian noise than median filter.
- Median filter is better at dealing with salt pepper noise than mean filter

Comparison Table:

Feature	Median Filter (Non-linear)	Mean Filter (Linear)
Operation	Replaces pixel with the middle value after sorting neighbors.	Replaces pixel with the average of neighbors.
Effect on Outliers	Ignores extreme values (0 or 255). Median of [0,100,255] = 100.	Pulled by extreme values. Mean of [0,100,255] = 118.
Edge Preservation	Preserves sharp edges because it doesn't create new pixel values (no new pixel values created).	Blurs edges creates smooth transitions
Best for Noise Type	Excellent for Salt-and-pepper noise (random white/black pixels).	Good for Gaussian noise (random variations around mean).

Q4. Describe the characteristics and drawbacks of Ideal Low Pass Filter (ILPF)

Answer: The Ideal Low Pass Filter (ILPF) is a frequency domain filter that passes all frequencies within a circle of radius D_0 (cutoff frequency) and rejects (cuts off) all frequencies outside this circle.

Characteristics:

- **Transfer Function**

- $H(u,v) = 1$ if $D(u,v) \leq D_0$ (pass)
- $H(u,v) = 0$ if $D(u,v) > D_0$ (reject)

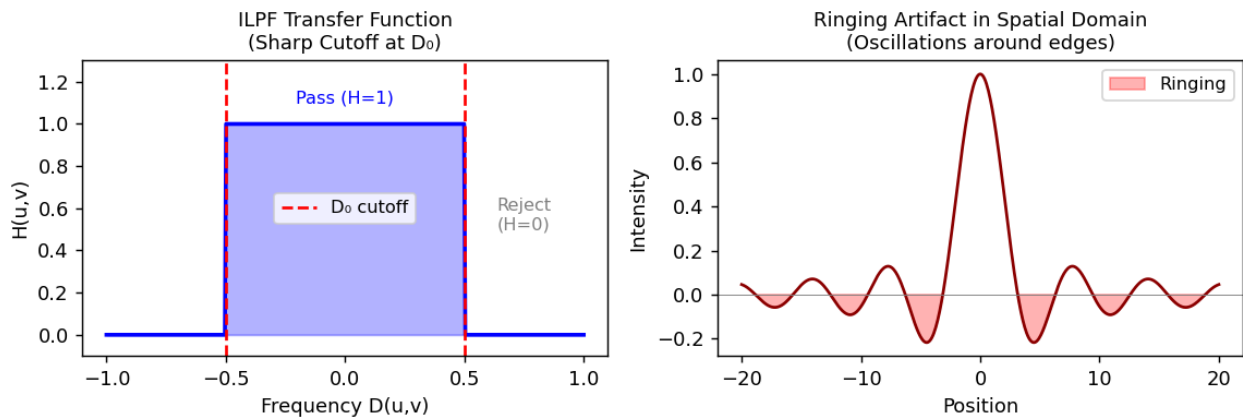
$D(u,v)$ = distance from center of frequency rectangle. D_0 = cutoff frequency.

- **Sharp Cutoff:** The transition between passing and rejecting frequencies is instantaneous (vertical).

Drawbacks:

- **1. Ringing Artifacts:** The primary and most severe drawback. The sharp cutoff in the frequency domain creates strong oscillations (rings) around edges in the spatial domain, resembling a bullseye pattern. This degrades image quality significantly.
- **2. Blurring:** Although it preserves low frequencies (which carry shape information), the removal of high frequencies inevitably blurs edges and fine details.
- **3. Not Physically Realizable:** The sharp transition is impossible to achieve with electronic components, making it only a theoretical concept.

Ideal Low Pass Filter (ILPF)



Q5. Generate the Filtered Image Using 3×3 Averaging Mask (Worked Example)

Sample Input Image (4×4 Matrix)

$$A = \begin{bmatrix} 12 & 18 & 24 & 30 \\ 10 & 16 & 22 & 28 \\ 14 & 20 & 26 & 32 \\ 8 & 12 & 18 & 24 \end{bmatrix}$$

3x3 averaging mask:

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Step 1: First 3x3 region

$$\begin{bmatrix} 12 & 18 & 24 \\ 10 & 16 & 22 \\ 14 & 20 & 26 \end{bmatrix}$$
$$= \frac{12 + 18 + 24 + 10 + 16 + 22 + 14 + 20 + 26}{9}$$
$$= \frac{162}{9} = 18$$

Step 2: Second 3x3 region

$$\begin{bmatrix} 18 & 24 & 30 \\ 16 & 22 & 28 \\ 20 & 26 & 32 \end{bmatrix}$$
$$= \frac{18 + 24 + 30 + 16 + 22 + 28 + 20 + 26 + 32}{9}$$
$$= \frac{216}{9} = 24$$

Step 3: Third 3x3 region

$$\begin{bmatrix} 10 & 16 & 22 \\ 14 & 20 & 26 \\ 8 & 12 & 18 \end{bmatrix}$$
$$= \frac{10 + 16 + 22 + 14 + 20 + 26 + 8 + 12 + 18}{9}$$
$$= \frac{146}{9} = 16.22$$

Step 4: Fourth 3x3 region

$$\begin{bmatrix} 16 & 22 & 28 \\ 20 & 26 & 32 \\ 12 & 18 & 24 \end{bmatrix}$$
$$= \frac{16 + 22 + 28 + 20 + 26 + 32 + 12 + 18 + 24}{9}$$
$$= \frac{198}{9} = 22$$

So, the final filtered image is:

$$G = \begin{bmatrix} 18 & 24 \\ 16.22 & 22 \end{bmatrix}$$

Q6. Describe suitable Filtering Techniques for Restoring a medical X-ray Image corrupted by Gaussian Noise

Answer: Gaussian noise is statistical noise with a probability density function equal to the normal distribution. It affects almost every pixel. For a medical X-ray image, preserving edges is critical for diagnosis. Suitable filtering techniques are:

1. **Mean Filter (Averaging Filter):** This is a linear filter that is effective at reducing Gaussian noise because the average of several random Gaussian values approaches the true mean. **Technique:** Use a small mask (e.g., 3×3 or 5×5). **Drawback:** It blurs edges, potentially obscuring fine details like small fractures.

2. **Gaussian Filter:** This is a weighted average filter where the weights are determined by a Gaussian function. Pixels closer to the center have higher weight. **Technique:** Convolve the image with a Gaussian kernel. **Advantage:** It provides a smoother, more natural-looking blur than the mean filter and is very effective for Gaussian noise while causing less ringing.

3. **Frequency Domain Filtering (Butterworth Low Pass Filter):** Gaussian noise has high-frequency components. A low pass filter can suppress them. **Technique:** Use a Butterworth Low Pass Filter (BLPF) instead of an Ideal LPF. Why BLPF? It has a smooth transition between passing and rejecting frequencies, which eliminates the ringing artifacts of the Ideal LPF.

Butterworth LPF Transfer Function

$$H(u, v) = 1 / [1 + (D(u, v) / D_0)^{2n}]$$

Where n = order (higher $n \rightarrow$ sharper cutoff), D_0 = cutoff frequency.

Conclusion for Medical X-ray: A **Gaussian filter** in the spatial domain or a **Butterworth Low Pass Filter** in the frequency domain is most suitable. These filters smooth the Gaussian noise effectively without introducing the severe ringing or blocky artifacts that could hide a critical medical finding.

Q7. Develop a Median Filtering Method for removing Salt-and-Pepper Noise from a corrupted image

Answer: To remove salt-and-pepper noise (random white and black pixels), a non-linear median filtering method is used. Here is the step-by-step development of the method:

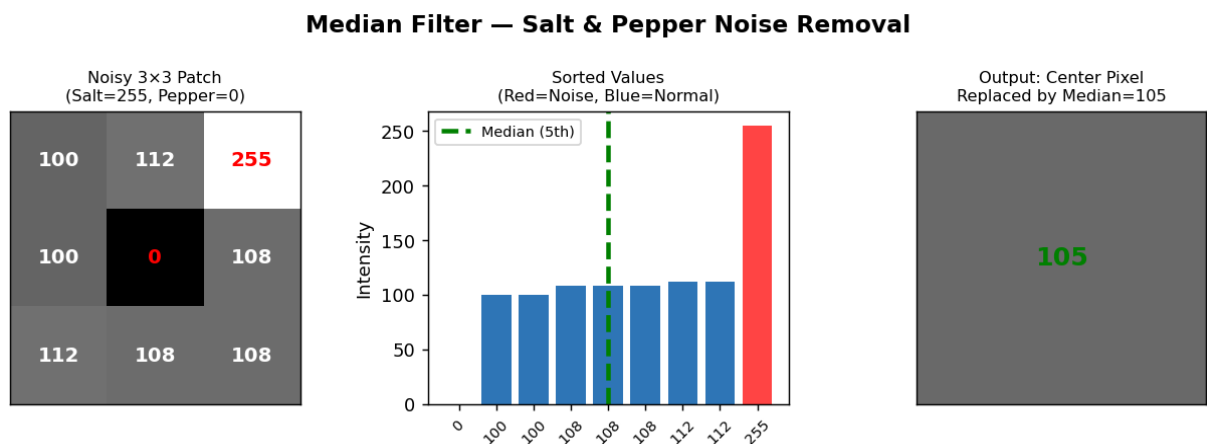
Problem: An image $f(x, y)$ is corrupted by salt-and-pepper noise. Some pixels are 0 (pepper, black) or 255 (salt, white), while others are uncorrupted.

Method Steps:

1. **Select a Filter Mask:** Choose a spatial neighborhood (mask). A 3×3 square mask is common and effective. For heavy noise, a larger mask (e.g., 5×5) can be used.
2. **Slide the Mask:** Slide the mask so that its center is positioned over each pixel (x, y) in the noisy image, moving one pixel at a time.
3. **Collect and Rank Pixels:** For the current position, collect all pixel intensity values within the mask. Sort these values in ascending order. For a 3×3 mask, there will be 9 values.
4. **Find the Median Value:** Identify the middle value in the sorted list. For a list of 9 numbers, the 5th value is the median.
5. **Replace the Center Pixel:** Replace the original noisy pixel value $f(x, y)$ with the median value. This becomes the output pixel $g(x, y)$.
6. **Repeat:** Move to the next pixel and repeat steps 2-5 until every pixel in the image has been processed.

Why this works: The noisy salt (255) and pepper (0) pixels are extreme values. When the list is sorted, these extreme values will always be at the beginning (0s) or the end (255s). The median, being the middle value, will

be a normal, uncorrupted pixel from the neighborhood. Thus, the noise is effectively removed without blurring edges.



Q8. Thresholding

Thresholding is the **simplest method for image segmentation**. It converts a grayscale image into a binary image by dividing pixels into 2 classes.

Thresholding is the operation of selecting a threshold value T and applying the following rule to each pixel $f(x, y)$ in the input image to produce an output binary image $g(x, y)$:

Formula

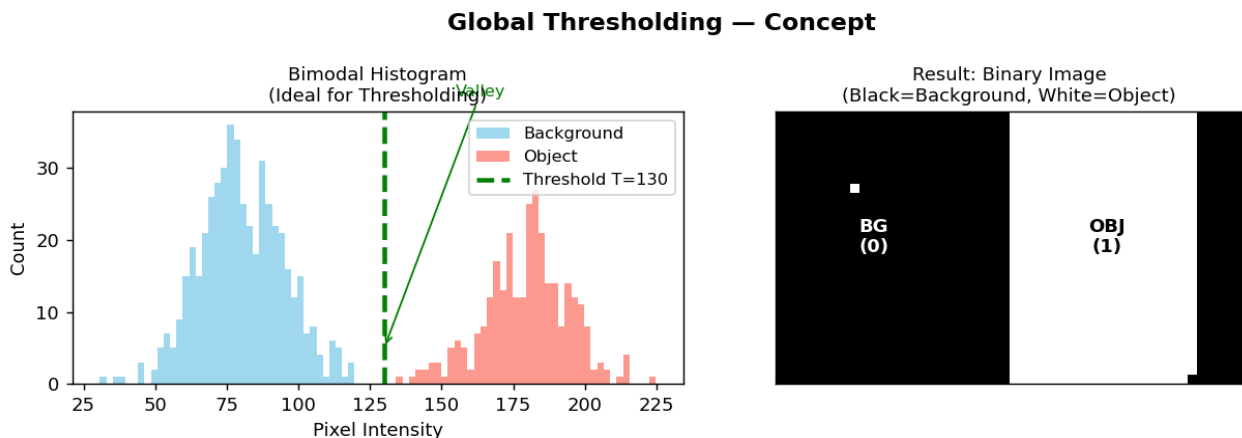
$$g(x, y) = 1 \quad (\text{White}) \quad \text{if } f(x, y) > T$$

$$g(x, y) = 0 \quad (\text{Black}) \quad \text{if } f(x, y) \leq T$$

Where **T = threshold value** and $f(x,y)$ = original pixel intensity.

Basic Principle

Thresholding works best on images with a **bimodal histogram** — two distinct peaks, one for background and one for objects. The threshold T is placed in the valley between the two peaks.



Q9. Basic Global Thresholding — Limitation

Answer: Global thresholding uses a **single, constant threshold value T** for the entire image. Its main limitation is that it fails completely when the image has **non-uniform illumination**.

Explanation of the Limitation: For global thresholding to work, the image must have a clear separation between background and object intensities — and this separation must remain consistent throughout the entire image.

However, in real life, lighting is rarely uniform. One part of the image may be brightly lit while another part may be in shadow. Because of this uneven illumination, the same threshold T that works well on the bright part of the image will not work on the dark part, and if you change T to fix the dark part, it will then break the bright part.

Example: Consider an image of a dark coin on a light table. If the left side of the table is brightly lit and the right side is in shadow:

- On the bright left side: Background is 200, Coin is 100. A threshold of 150 works.
- On the shadowed right side: Background is 120, Coin is 50. A threshold of 150 would make both the coin and the background black (incorrect). A threshold of 80 would make the background white on the right, but the coin white on the left.

Example — Coin on Non-Uniform Table

Region	Background Intensity	Coin Intensity	Result with T=150
Bright Left Side	200	100	✓ Works correctly ($100 < 150 < 200$)
Shadow Right Side	120	50	X FAILS — both coin and background become black (both < 150)

Conclusion:

No single value of T can correctly segment the entire image when illumination varies across the scene. This is the fundamental limitation of global thresholding — one threshold cannot handle non-uniform lighting conditions.

Q10. Design an adaptive thresholding technique to segment a grayscale image (include diagram).

Answer: Adaptive thresholding (also called local thresholding) overcomes the limitation of global thresholding by computing a different threshold value for each pixel, based on the content of its local neighborhood.

Design Steps:

1. **Divide into Local Regions:** For each pixel (x, y) in the input grayscale image, consider a small neighborhood window (e.g., 11×11 or 15×15 pixels) centered on that pixel.
2. **Compute Local Statistics:** From the pixels within that window, calculate a local statistic. The most common is the **local mean** $m(x, y)$. Sometimes the **local median** or a weighted average is used.
3. **Compute the Local Threshold:** Determine the threshold $T(x, y)$ for the center pixel using a formula based on the local statistic. A simple and effective formula is:
$$T(x, y) = m(x, y) - C$$
where $m(x, y)$ is the local mean and C is a constant.
4. **Apply the Threshold:** Compare the original pixel value $f(x, y)$ with its own local threshold $T(x, y)$ to produce the output binary pixel $g(x, y)$:

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > T(x,y) \\ 0 & \text{otherwise} \end{cases}$$

5. **Repeat:** Repeat this process for every pixel (x , y) in the image.

Why it works: In a dark region, the local mean m (x , y) will be low, so the threshold T (x , y) will also be low, allowing details to be separated from the dark background. In a bright region, m (x , y) is high, so T (x , y) is high as well. Thus, the threshold adapts to the local illumination.

Q11. Image Segmentation — Definition & Importance

Definition: Image segmentation is the process of partitioning a digital image into multiple meaningful regions or sets of pixels (called segments). Each segment represents a distinct object, part of an object, or region of interest within the image. The goal is to simplify or change the representation of an image into something more meaningful and easier to analyze.

Importance in Different Fields

Field	Application
Healthcare	Segments organs, tumors, or cells in MRI/CT/X-ray for diagnosis and treatment planning.
Autonomous Vehicles	Segments road, pedestrians, other vehicles, and traffic signs from camera feeds.
Industrial Inspection	Detects defects, chips, or cracks on manufactured parts by analyzing boundaries.
Object Detection	Segmentation simplifies the image, allowing algorithms to focus on the shape and boundaries of objects rather than processing every pixel individually.
Biology	Segments cells, organisms, or biological structures to study their morphology.
OCR / Character Recognition	Segments individual characters from documents for recognition.
Finance	Identifies customer segments for customized financial products, fraud detection (segmenting normal vs. anomalous transactions), and risk management.

Q12. Build a Region-Based Segmentation process for a given image (explain Step-by-Step)

Answer: We will build the **Region Growing** algorithm, a classic region-based segmentation method. It starts from seed points and grows regions by merging neighboring pixels with similar properties.

Algorithm Steps

1. **Select Seed Points:** Choose one or more starting pixels (seeds) inside the desired regions. Seeds can be selected manually by a user or automatically (e.g., using pixel intensity thresholds or morphological processing).

Example: To segment a tumor in an MRI, a doctor clicks inside the tumor area.

2. **Define Similarity Criterion:** Define a rule: e.g., intensity must be within tolerance T of the region's mean.

Criterion: $|f(x,y) - \text{Mean of region}| < \text{Threshold}$

3. **Initialize the Region:** Create output image g (same size). Set all pixels to 0 (background). Mark seed pixels as 1 (foreground).
4. **Grow the Region:** Examine all unlabeled adjacent pixels (4-connected or 8-connected) to the current region. If a neighbor satisfies the similarity criterion, add it to the region and update the mean.
5. **Repeat Until Convergence:** Continue until no more pixels can be added to any region.
6. **Post-processing (Optional):** Merge very small regions with larger neighbors to remove tiny artifacts.

Diagram:

Step 1: Seed		Step 2: Iteration 1		Step 3: Iteration 2		Step 4: Final Region
[. . . .]		[. . . .]		[. . . .]		[S S S .]
[. S . .]	-->	[. S S .]	-->	[. S S S]	-->	[S S S S]
[. . . .]		[. . . .]		[. S . .]		[S S S .]
[. . . .]		[. . . .]		[. . . .]		[. . . .]

(S = Seed, S = Grown region) (Similarity criterion = intensity close to 100)

Iteration	Region Pixels Added	Condition
Seed	(row 1, col 1)	Manual seed placed
Iteration 1	(1,2), (2,1)	Intensity close to seed
Iteration 2	(1,3), (2,2), (3,1)	Within tolerance of current mean
Final	All similar connected pixels	No more pixels to add → stop

Q13. Define Structuring Element, Erosion, and Dilation

1. Structuring Element (SE)

A structuring element is a small binary matrix or "probe" used to examine and interact with the pixels of an image in morphological processing. It defines the neighborhood shape and size for operations like erosion and dilation. It has an **origin** (usually the center pixel) that is placed over each pixel of the image. Common shapes include 3×3 square, cross, or line.

Diagram:

3x3 Square SE	Cross SE	Origin (●)
[1 1 1]	[0 1 0]	
[1 1 1]	[1 1 1]	
[1 1 1]	[0 1 0]	

Shape	Matrix	Common Use
3x3 Square	[1,1,1 / 1,1,1 / 1,1,1]	General morphological ops (erosion/dilation)
Cross / Plus	[0,1,0 / 1,1,1 / 0,1,0]	4-connected neighborhood operations

2. Erosion ($A \ominus B$)

Erosion shrinks or thins objects in a binary image. For a binary image A and structuring element B, erosion $A \ominus B$ is defined as the set of all points where B, translated to that point, **completely fits** inside A.

Rule: A pixel in the output is set to 1 **ONLY IF** all '1' pixels in the structuring element overlap with '1' pixels in the image.

$A \ominus B = \{ z \mid (B)_z \subseteq A \}$ (B translated to z must completely fit inside A)

Effect: Removes small objects, separates touching objects, shrinks region boundaries.

3. Dilation ($A \oplus B$)

Dilation grows or expands objects in a binary image. For a binary image A and structuring element B, dilation $A \oplus B$ is defined as the set of all points where B, translated to that point, **hits** or overlaps with A.

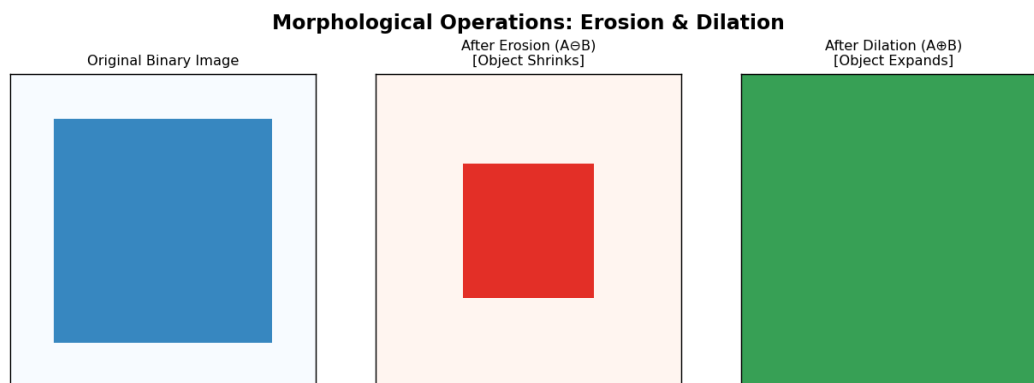
Rule: A pixel in the output is set to 1 **IF** at least one '1' pixel in the structuring element overlaps with a '1' pixel in the image.

$A \oplus B = \{ z \mid (B)_z \cap A \neq \emptyset \}$ (B translated to z must hit at least one pixel of A)

Effect: Repairs broken objects, fills small holes and gaps, expands region boundaries.

Comparison Diagram:

Original Binary Image		After Erosion		After Dilation
[0 0 0 0 0]		[0 0 0 0 0]		[0 0 0 0 0]
[0 1 1 1 0]		[0 0 1 0 0]		[0 1 1 1 0]
[0 1 1 1 0]	with 3x3	[0 0 1 0 0]	with 3x3	[0 1 1 1 0]
[0 1 1 1 0]	SE -->	[0 0 1 0 0]	SE -->	[0 1 1 1 0]
[0 0 0 0 0]		[0 0 0 0 0]		[0 0 0 0 0]
		(Object Shrinks)		(Object Expands)



Q14. Describe the Applications of Boundary Extraction in DIP

Formula

$$\beta(A) = A - (A \ominus B)$$

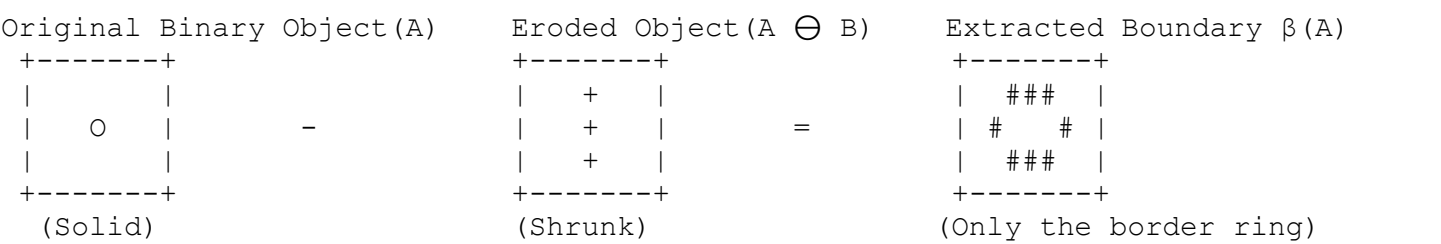
Answer: Boundary extraction (also called contour extraction) is a morphological algorithm that finds the outline of objects in a binary image. The formula is: $\beta(A) = A - (A \ominus B)$, where B is a structuring element. By subtracting the eroded image (which has a thinner object) from the original image, only the outer ring of pixels (the boundary) remains.

Applications

1. Object Recognition & Shape Analysis: The boundary is the most distinctive feature of an object's shape. Extracting the boundary simplifies the object's representation from a solid shape to a 1-pixel thick contour, making it easier to compute shape descriptors like perimeter, curvature, and moments for object classification.

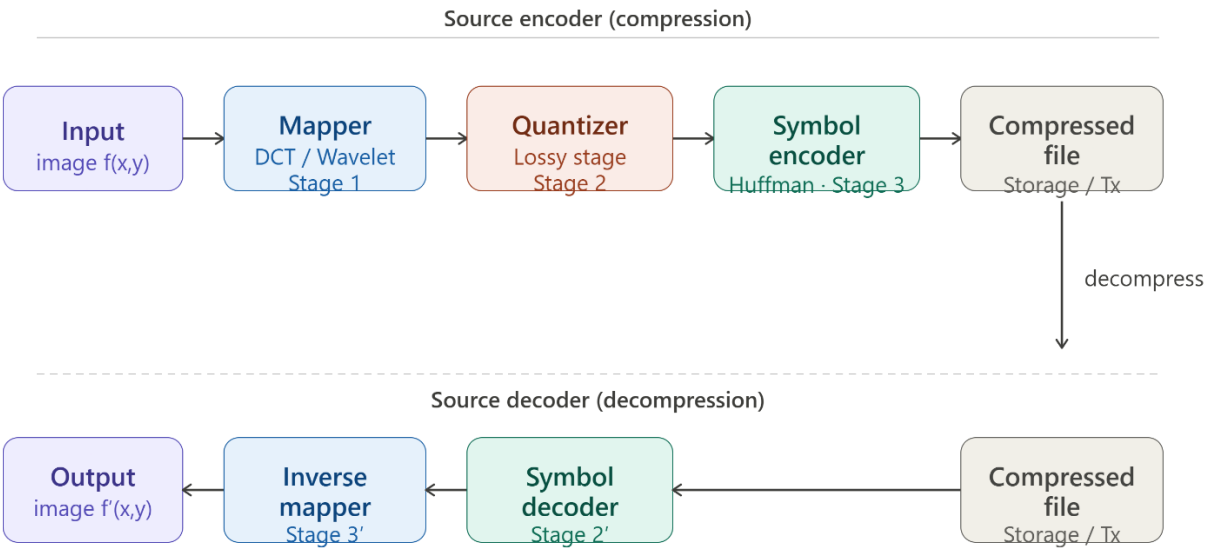
2. **Medical Imaging (Cell & Organ Analysis):** Used to extract the exact boundaries of cells, nuclei, or organs from segmented microscopy or MRI images. This allows pathologists to measure cell size, shape irregularity, and nuclear-cytoplasmic ratio, which are critical for diagnosing diseases like cancer.
3. **Industrial Inspection & Quality Control:** Used to find the outer contour of manufactured parts. The extracted boundary can be compared to a perfect template to detect defects such as chips, cracks, or deformations on the edges of products.
4. **Character Recognition (OCR):** After segmenting text from a document, boundary extraction can be used as a preprocessing step to find the contour of each character. This simplifies the character's structure for feature extraction (e.g., loops, endpoints, intersections) before classification.
5. **Image Registration & Matching:** Boundaries provide a robust feature for aligning two images of the same scene (e.g., satellite or medical images). Matching the extracted boundaries is often faster and more reliable than matching all pixels.

Diagram:



Q15. Design an Image Compression Model and explain its major components

Answer: An image compression model is a system that reduces the amount of data required to represent an image. It consists of a **source encoder** (for compression) and a **source decoder** (for decompression).



A. Source Encoder — 3 Stages

1. **Mapper (Stage 1 — Transformation):** Transforms input image to reduce interpixel redundancy. Techniques: DCT (used in JPEG), Wavelet transforms, Differential Encoding (store differences between adjacent pixels).

- 2. **Quantizer (Stage 2 — Lossy):** The ONLY lossy stage. Reduces precision based on what the human eyes can't perceive (psychovisual redundancy). High-frequency DCT coefficients are rounded to zero.
- 3. **Symbol Encoder (Stage 3 — Coding):** Creates binary code for quantized values. Shorter code words for more frequent values. Example: Huffman Coding. Reduces coding redundancy WITHOUT losing data.

B. Source Decoder — 2 Stages

- 1. **Symbol Decoder:** Converts compressed binary stream back to quantized values (inverse of Symbol Encoder).
- 2. **Inverse Mapper:** Applies inverse transformation to reconstruct image data. Some data lost due to quantization (lossy compression).

Q16. Different Thresholding Methods

Method	Description	When to Use	Formula / Key Idea
1. Global Thresholding	Single constant T applied to every pixel of the entire image.	Images with bimodal histogram & uniform illumination.	$g(x,y) = 1 \text{ if } f(x,y) > T$
2. Adaptive / Local Thresholding	$T(x,y)$ changes per pixel based on local neighborhood.	Non-uniform illumination, shadows, varying contrast.	$T(x,y) = m(x,y) - C$
3. Otsu's Method	Automatic method that finds T minimizing intra-class variance between foreground & background.	Bimodal histogram where optimal T is unknown.	Minimize: $\sigma^2_{\text{within}}(T) = P_1(T)\sigma_1^2 + P_2(T)\sigma_2^2$
4. Multiple Thresholding	Uses more than one threshold to create more than two classes (dark, medium, bright).	Images with multiple objects at different intensity ranges.	$g(x,y) = 1 \text{ if } T_1 < f(x,y) \leq T_2$

Q17. Types of Data Redundancy

Data redundancy is unnecessary repetition or predictable data that can be removed without losing essential information. Image compression exploits three main types.

Type	Definition	Compression Approach	Example
1. Coding Redundancy	Fixed-length codes (8 bits) are wasteful when some intensities occur far more than others.	Variable Length Coding (VLC) — shorter codes for frequent values.	Huffman Coding: Intensity 100 (80% pixels) → code '1'; Intensity 255 (2%) → '00101'

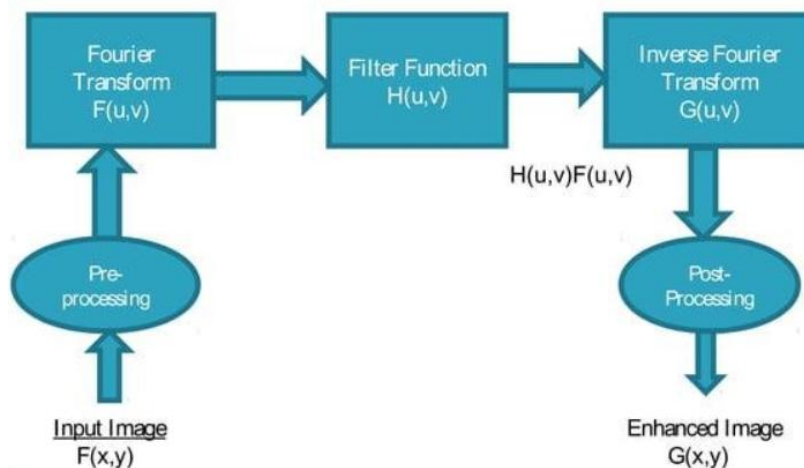
Type	Definition	Compression Approach	Example
2. Interpixel Redundancy (Spatial/Temporal)	Adjacent pixels are highly correlated — a pixel value can be predicted from neighbors.	Transform coding (DCT, Wavelet) or Differential Encoding to decorrelate pixels.	Store [100,1,1,1] instead of [100,101,102,103] using differences.
3. Psychovisual Redundancy	The human visual system (HVS) is less sensitive to fine color detail and high-frequency changes.	Quantization — reduce precision of information HVS can't perceive.	JPEG: High-frequency DCT coefficients (fine detail) divided by large number and rounded to 0.

Q18. Describe the process of filtering in the frequency domain with necessary diagrams.

Filtering in the frequency domain involves modifying the Fourier transform of an image to enhance or suppress specific features (like noise or edges). The process relies on the **Convolution Theorem**, which states that convolution in the spatial domain is equivalent to multiplication in the frequency domain.

The Process

1. **Fourier Transform:** Convert the input image $f(x, y)$ from the spatial domain to the frequency domain using the 2D Fast Fourier Transform (FFT) to get $F(u, v)$
2. **Filter Design:** Create a filter function $H(u, v)$ that defines which frequencies to keep (pass-band) or remove (stop-band).
3. **Multiplication:** Multiply the image spectrum by the filter: $G(u, v) = F(u, v) \times H(u, v)$
4. **Inverse Transform:** Apply the Inverse 2D FFT to $G(u, v)$ to return the image to the spatial domain, resulting in the filtered output $g(x, y)$.



Q19. Describe the process of image restoration.

Image restoration aims to reconstruct or recover an image that has been degraded by a known or estimated degradation process. Unlike enhancement (which is subjective), restoration is an **objective process** based on mathematical models of the degradation.

Key Components

- **Degradation Model:** Represented as $g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$, where f is the original image, h is the degradation function (e.g., motion blur), $*$ is convolution, and η (eta) is additive noise.
- **Estimation:** The goal is to estimate the original image by $\hat{f}$ applying an inverse filter or a restoration filter, such as:
 - **Inverse Filtering:** Simply dividing by the degradation function (highly sensitive to noise).
 - **Wiener Filtering:** A statistical approach that balances noise reduction with inverse filtering to minimize the mean square error.
 - **Constrained Least Squares Filtering:** Useful when noise statistics are not fully known.

Q20. Describe the importance of image segmentation in medical and industrial applications.

Image segmentation is the process of partitioning an image into multiple segments (sets of pixels) to simplify its representation into something more meaningful and easier to analyze.

Medical Applications

Segmentation is critical for **computer-aided diagnosis and surgical planning**:

- **Anatomical Analysis:** Isolating organs (heart, liver, lungs) from CT or MRI scans to calculate volume or detect abnormalities.
- **Tumor Detection:** Precisely outlining the boundaries of a lesion or tumor to assess its size and growth over time, which is essential for radiotherapy planning.
- **Tissue Classification:** Differentiating between grey matter, white matter, and cerebrospinal fluid in brain imaging.

Industrial Applications

Segmentation is the backbone of **automated quality control and robotics**:

- **Defect Detection:** Isolating components on a production line to identify scratches, cracks, or missing parts that deviate from a "golden" template.
- **Robotic Vision:** Enabling robots to identify, locate, and grip specific parts in unstructured environments (e.g., bin picking).
- **Measurement:** Automatically segmenting an object to calculate its dimensions (area, diameter) to ensure it meets manufacturing tolerances.